# 两相步进FOC基于spwm算法复现心得

## FOC三环控制框架

在旋转运动中，输入位置指令，这个指令可以是rad圈数，那么单位就是rad，经过三环之后输出为电机扭矩（电流大小）单位Nm。在这个过程中，位置给定和位置反馈比较值成为位置误差，当量单位是rad，当位置误差与位置环增益相乘，计算结果给到速度环，单位是rad/s，所以位置环增益的当量单位为1/s，但是速度给定与速度反馈比较后的速度误差单位也是1/s，到最后输出Nm是如何转换的呢

1. **位置**：表示在某一特定时刻物体相对于参考点的角位移，单位通常是弧度（rad）。
2. **速度**：位置的一阶导数，表示位置随时间的变化率，单位是弧度每秒（rad/s）。

### 从位置到速度的转换

在控制系统中，位置环的输出需要控制电机的速度。位置环输出的基本公式为：

$$\text{速度设定}(\mathrm{rad/s}) = K_p \times \text{位置误差}(\mathrm{rad})$$

其中 $K_p$ 是位置环增益。

- **位置误差**（单位为 $\mathrm{rad}$）乘以位置环增益 $K_p$（单位为 $1/\mathrm{s}$）得到的结果是速度设定值（单位为 $\mathrm{rad/s}$）。

### 单位转换

- 当我们说增益的单位为 $1/\mathrm{s}$，这表示增益将输入的每弧度位置误差转换为每秒弧度的速度变化。
- 这样，增益实质上是一个时间的倒数（$1/\mathrm{s}$），描述了位置变化到速度变化的转换速率。

### 解释为什么 $\mathrm{rad/s}$ 转换为 $1/\mathrm{s}$

- 在控制论和物理中，经常根据所需的控制作用和物理意义来定义单位。在这里，虽然速度的直接单位是 $\mathrm{rad/s}$，但增益的单位 $1/\mathrm{s}$ 实际上是从角度单位（rad）转换为速度单位（rad/s）的系数。
- 因此，当我们将位置误差（rad）乘以位置环增益（$1/\mathrm{s}$），其结果自然是速度（rad/s），说明这种单位转换实际上反映了控制动作的物理意义和实现机制。

总结来说，这种单位转换反映了控制系统设计中如何将位置误差（一个角位移）通过控制增益转化为速度指令（角速度）。这是一种将时间维度（速率）与空间维度（位移）联系起来的方法，是控制理论中常见的做法。

转矩的当量 1N= 1kg m/s²

1Nm = 1kg m/s² *m

所以1Nm = 1kg m²/s²

要使得速度单位 1/s 运算到这个转矩的单位1kg m²/s² 需要与两个当量单位作乘积

一个是1/s ，这个与时间频率有关，是速度环增益的单位，另外一个是kg m²这个是惯量的单位，是从电流环给定到电机扭矩输出的传递系数。

速度环与位置环的增益系数都是1/s为单位，反应去东区对速度和位置的动态响应

## 电流环中，电流给定到输出的传递系数单位和惯量单位一样，说明?

三环控制过程，在位置环，将位置误差进行求导，那么就是d/dt,从而得出速度的参考指令，自然位置环的增益系数是1/s，在速度环将速度指令（速度误差）值进行求导，那么就是d/dt从而得出当前轴的加速度，那么速度环的增益系数单位也是1/s，那么从速度环给到电流环的值，就是该轴的加速度，单位是1/s²，从加速度值到转矩，可以用牛顿第二定律联系，也就是上文提到转矩

M=Iβ

$$M = I\beta$$

M：转矩（Nm）

I：转动惯量✦（kg · m2）

β：加速度（1/s2）

由此三环得以联系起来，可以通过三级PID串联，得出电流控制值，也是可以获得转矩控制量。

# 常规基础配置

首先是要与磁编芯片进行通讯，本文磁编芯片采用的是MT6835是用spi进行通讯，通讯时要注意spi模式，第几个有效和上升沿与下降沿，代码示例

/**

- @brief  SPI2接口引脚配置
- @param  GPIO_Pin_12 GPIO_Pin_13 GPIO_Pin_14 GPIO_Pin_15
- @return
- @example
- *
- 注意：
  /
  *static void bsp_SPIGpioConfig(void)*
  *{*
  *GPIO_InitTypeDef GPIO_InitStructure;*
  / 使能 ADIS_SPI 及GPIO 时钟 */
  RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB,ENABLE);
  RCC_APB1PeriphClockCmd(MT6835_SPI_CLK, ENABLE);

/* 设置引脚复用 */

```
GPIO_PinAFConfig(MT6835_SPI_CLK_GPIO_PORT,MT6835_SPI_CLK_PINSOURCE,GPIO_AF_SPI2);
GPIO_PinAFConfig(MT6835_SPI_MISO_GPIO_PORT,MT6835_SPI_MISO_PINSOURCE,GPIO_AF_SPI2
);
GPIO_PinAFConfig(MT6835_SPI_MOSI_GPIO_PORT,MT6835_SPI_MOSI_PINSOURCE,GPIO_AF_SPI2
);
```

*/配置复用输出SPI2的CS参数/*
GPIO_InitStructure.GPIO_Pin = MT6835_SPI_CS_PIN;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Mode =  GPIO_Mode_OUT ;

```
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(MT6835_SPI_CS_GPIO_PORT,&GPIO_InitStructure);

/配置复用输出SPI2的SCK参数/
    GPIO_InitStructure.GPIO_Pin = MT6835_SPI_CLK_PIN;
    GPIO_InitStructure.GPIO_Mode =  GPIO_Mode_AF ;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_Init(MT6835_SPI_CLK_GPIO_PORT,&GPIO_InitStructure);

/配置复用输出SPI2的MISO参数/
    GPIO_InitStructure.GPIO_Pin = MT6835_SPI_MISO_PIN;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(MT6835_SPI_MISO_GPIO_PORT,&GPIO_InitStructure);

/配置复用输出SPI2的MOSI参数/
    GPIO_InitStructure.GPIO_Pin = MT6835_SPI_MOSI_PIN;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_DOWN;
    GPIO_Init(MT6835_SPI_MOSI_GPIO_PORT,&GPIO_InitStructure);

}

/**

- @brief  SPI功能模式配置
- @param  GPIO_Pin_12 GPIO_Pin_13 GPIO_Pin_14 GPIO_Pin_15
- @return
- @example
-  *
- 注意：
  */

void bsp_SPIConfig(void)
{
    SPI_InitTypeDef  SPI_InitStructure;
    bsp_SPIGpioConfig();
/配置SPI模式，MT6835支持空闲时刻高电平，第二个边沿采样 CPOL=1 CPHA=1/
    SPI_InitStructure.SPI_Direction = SPI_Direction_2Lines_FullDuplex;
    SPI_InitStructure.SPI_Mode = SPI_Mode_Master;
    SPI_InitStructure.SPI_DataSize = SPI_DataSize_8b;
    SPI_InitStructure.SPI_CPOL = SPI_CPOL_High;
    SPI_InitStructure.SPI_CPHA = SPI_CPHA_2Edge;
    SPI_InitStructure.SPI_NSS = SPI_NSS_Soft;
    SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_256;
    SPI_InitStructure.SPI_FirstBit = SPI_FirstBit_MSB;
    SPI_InitStructure.SPI_CRCPolynomial = 7;
    SPI_Init(SPI2,&SPI_InitStructure);
/开启SPI通讯/
    SPI_Cmd(SPI2,ENABLE);

}
```

uint8_t bsp_SPISwapByte(uint8_t ByteSend)
{
uint32_t timeout = 10000; // 超时计数器
while (SPI_I2S_GetFlagStatus (SPI2,SPI_I2S_FLAG_TXE) != SET)
{
if (--timeout == 0) return 0xFF; // 超时处理，返回错误码
}
SPI_I2S_SendData(SPI2,ByteSend);

```
while (SPI_I2S_GetFlagStatus (SPI2,SPI_I2S_FLAG_RXNE) != SET)
{
    if (--timeout == 0) return 0xFF; // 超时处理，返回错误码
}
return SPI_I2S_ReceiveData(SPI2);
```

}

标准库的SPI通讯指令往往带来很多问题，需要每个寄存器都核实，每个指令都要用到

配置完SPI要根据指令帧进行数据合并发生与读取数据，转换成自己有用的值，这部分需要参考手册

## 其次是ADC采样部分

在ADC采样，要明白FOC流程，首先是高级定时器输出4个通道，用的中间对齐模式，那么repeattion这个值需要给2，因为在第二个有效事件触发注入通道的采样，用注入通道的采样，进入ADC中断完成FOC运算。其他不关键的ADC采样需要用规则组，然后在中断里面多次调用软启动即可，ADC的采样往往搭配着DMA的使用

void bsp_ADC1Config(void)
{
//开启ADC1采样通道时钟

```
bsp_ADC1GpioConfig();
ADC_CommonInitTypeDef ADC_CommonInitStructure;
ADC_InitTypeDef ADC_InitStructure;
RCC_APB2PeriphClockCmd(VOLTAGE_ADC_CLK1,ENABLE);
```

//复位ADC时钟通道
ADC_DeInit();

*/ADC模式设置，采用独立模式，不采用DMA转运，时钟是4分频，间隔是4个ADC采样与103有出入/*
ADC_CommonStructInit(&ADC_CommonInitStructure);
ADC_CommonInitStructure.ADC_DMAAccessMode = ADC_DMAAccessMode_Disabled;
ADC_CommonInitStructure.ADC_Mode = ADC_Mode_Independent;
ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div4;
ADC_CommonInitStructure.ADC_TwoSamplingDelay =ADC_TwoSamplingDelay_5Cycles;
//两个采样阶段下，多通道下模式有效
ADC_CommonInit(&ADC_CommonInitStructure);
//ADC结构体参数赋值
ADC_StructInit(&ADC_InitStructure);
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE; //关闭连续转运，使用定时器触发

ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right; //右对齐模式
// 软件触发不用选取ADC_InitStructure.ADC_ExternalTrigConv
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None;//默认值
ADC_InitStructure.ADC_NbrOfConversion = 3; //3个序列转换
ADC_InitStructure.ADC_Resolution= ADC_Resolution_12b;//有效性12位
ADC_InitStructure.ADC_ScanConvMode = ENABLE;//扫描模式
ADC_Init(VOLTAGE_ADC1, &ADC_InitStructure);

```
bsp_ADCRegularChannelConfiguration();
bsp_ADCInjectChannelConfiguration();
bsp_ADC1NvicConfig();

ADC_ExternalTrigInjectedConvConfig(ADC1, ADC_ExternalTrigInjecConv_T1_TRGO);
ADC_ExternalTrigInjectedConvEdgeConfig(ADC1,
ADC_ExternalTrigInjecConvEdge_Rising);

ADC_Cmd(VOLTAGE_ADC1,ENABLE);
ADC_DMACmd(VOLTAGE_ADC1, ENABLE);
ADC_DMARequestAfterLastTransferCmd(VOLTAGE_ADC1, ENABLE);//开启DMA连续转运
```

}

注入组

static void bsp_ADCInjectChannelConfiguration(void)
{
#define SampleTime_Injected ADC_SampleTime_3Cycles

```
/* ADC1/2 injected channel Configuration */
ADC_InjectedSequencerLengthConfig(ADC1, 2);

//-----------------------------------------------------------------//
ADC_InjectedChannelConfig(ADC1, AVOLTAGE_ADC_CHANNEL, 1, SampleTime_Injected);
ADC_InjectedChannelConfig(ADC1, BVOLTAGE_ADC_CHANNEL, 2, SampleTime_Injected);
```

```
//-----------------------------------------------------------------//
//ADC_ExternalTrigInjectedConvConfig(ADC1, ADC_ExternalTrigInjecConv_T1_TRGO);

//ADC_ExternalTrigInjectedConvEdgeConfig(ADC1,
ADC_ExternalTrigInjecConvEdge_None);
```

```
/* Disable automatic injected conversion start after regular one */
ADC_AutoInjectedConvCmd(ADC1, DISABLE);
```

}

规则组

```
static void bsp_ADCRegularChannelConfiguration(void)
{
    //使能内部温度传感器
    ADC_TempSensorVrefintCmd(ENABLE);
    #define     SampleTime_Regular      ADC_SampleTime_112Cycles
    ADC_RegularChannelConfig(VOLTAGE_ADC1, BUSVOLTAGE_ADC_CHANNEL, 1,
SampleTime_Regular);
    ADC_RegularChannelConfig(VOLTAGE_ADC1, ADC_Channel_TempSensor, 2,
SampleTime_Regular);
    ADC_RegularChannelConfig(VOLTAGE_ADC1, ADC_Channel_Vrefint, 3, SampleTime_Regular);

}

/**
```

- @brief  ADC1中断配置
- @param
- @return
- @example
- *
- 注意：
  */
  static void bsp_ADC1NvicConfig(void)
  {
  NVIC_InitTypeDef NVIC_InitStructure;  //中断配置
  ADC_ClearFlag(ADC1, ADC_FLAG_JEOC);
  //NVIC中断服务配置
  NVIC_InitStructure.NVIC_IRQChannel =ADC_IRQn;
  NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
  NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority =0;
  NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
  NVIC_Init(&NVIC_InitStructure);

  ADC_ITConfig(ADC1, ADC_IT_JEOC, DISABLE);
  }

TIM8高级定时器的配置

在配置的时候，需要启用触发ADC采样，否则无法进入中断

定时器四个通道都要开启，在每次中断后改CCR比较值

```
 void bsp_Tim8Init(void)
 {
```

```
TIM_OCInitTypeDef TIM_OCInitStructure;  //输出比较配置
TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
//NVIC_InitTypeDef NVIC_InitStructure;  //中断配置
bsp_Tim8GpioConfig();                   //启用GPIO
RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM8,ENABLE);

TIM_TimeBaseStructInit(&TIM_TimeBaseStructure);
TIM_TimeBaseStructure.TIM_ClockDivision = 0;
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_CenterAligned1;//中心对称模式1 产生下溢事件
TIM_TimeBaseStructure.TIM_RepetitionCounter = 1;
```

// TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;
// TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;//向上计数
TIM_TimeBaseStructure.TIM_Period = 3360-1 ;//周期 1Mhz / 10us*100 =1ms 中央对齐模式默认-1
TIM_TimeBaseStructure.TIM_Prescaler = 1-1 ;//APB1时钟是84M但是分到时钟X2 168000000/168 = 1Mhz
TIM_TimeBaseInit(TIM8,&TIM_TimeBaseStructure);

```
TIM_OCStructInit(&TIM_OCInitStructure);
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2;
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Disable;
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High; //有效电平是高电平，保证周期中间是高电平
TIM_OCInitStructure.TIM_OCNPolarity = TIM_OCNPolarity_High;
TIM_OCInitStructure.TIM_OCIdleState = TIM_OCNIdleState_Reset;
TIM_OCInitStructure.TIM_OCNIdleState = TIM_OCNIdleState_Reset;
```

```
//影子寄存器使能/预装载寄存器使能
TIM_OCInitStructure.TIM_Pulse =  TIM8_CH1_PWM_CCR;//CCR的值
TIM_OC1PreloadConfig(TIM8,TIM_OCPreload_Enable);
TIM_OC1Init(TIM8,&TIM_OCInitStructure);

TIM_OCInitStructure.TIM_Pulse =  TIM8_CH2_PWM_CCR;//CCR的值
TIM_OC2PreloadConfig(TIM8,TIM_OCPreload_Enable);
TIM_OC2Init(TIM8,&TIM_OCInitStructure);

TIM_OCInitStructure.TIM_Pulse =  TIM8_CH3_PWM_CCR;//CCR的值
TIM_OC3PreloadConfig(TIM8,TIM_OCPreload_Enable);
TIM_OC3Init(TIM8,&TIM_OCInitStructure);

TIM_OCInitStructure.TIM_Pulse =  TIM8_CH4_PWM_CCR;//CCR的值
TIM_OC4PreloadConfig(TIM8,TIM_OCPreload_Enable);
TIM_OC4Init(TIM8,&TIM_OCInitStructure);

TIM_ClearFlag(TIM8,TIM_FLAG_Update);
//NVIC中断服务配置
```

// NVIC_InitStructure.NVIC_IRQChannel = TIM8_UP_TIM13_IRQn;
// NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
// NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority =3;
// NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;

// NVIC_Init(&NVIC_InitStructure);
//使能中断
//TIM_ITConfig(TIM8,TIM_IT_Update,DISABLE);
TIM_ARRPreloadConfig(TIM8,ENABLE);
TIM_CtrlPWMOutputs(TIM8,ENABLE);
TIM_SetCounter(TIM8, 0);
TIM_Cmd(TIM8,DISABLE);

}

## 外部编码器ABZ信号，AB信号用编码器模式，Z信号用外部中断

这点没什么好说的，外部编码器记得调极性确定电机转的方向与编码器增的值方向一致就行

/**

- @brief TIM与输入捕获功能模式设置
- @param GPIO_Pin_0 GPIO_Pin_1
- @return
- @example
- *
- 注意：
  */

void bsp_TIM2Config(void)
{
TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
TIM_ICInitTypeDef TIM_ICInitstructure;

```
bsp_Tim2GpioConfig();
```

*/配置TIM模式/*
TIM_TimeBaseStructInit(&TIM_TimeBaseInitStructure );
// TIM_InternalClockConfig()不需要配置定时器由内部时钟驱动，内部时钟分频，向上计数也不需要
TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1 ;
TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
TIM_TimeBaseInitStructure.TIM_Period =4000-1 ;//周期的意思，表示ARR自动重装值

```
TIM_TimeBaseInitStructure.TIM_Prescaler = 1-1;  //预分配PSC,没有分频系数
TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;

TIM_TimeBaseInit(TIM2,&TIM_TimeBaseInitStructure);
```

*/配置IC模式,采用编码器模式，通道1与通道2/*
//赋默认值给IC结构体
TIM_ICStructInit(&TIM_ICInitstructure);
TIM_ICInitstructure.TIM_Channel = TIM_Channel_1;
TIM_ICInitstructure.TIM_ICFilter = 0xF;

//极性是否反转，并不是高低电平是否有效
TIM_ICInitstructure.TIM_ICPolarity =TIM_ICPolarity_Rising;
TIM_ICInit(TIM2,&TIM_ICInitstructure);

```
TIM_ICInitstructure.TIM_Channel = TIM_Channel_2;
TIM_ICInitstructure.TIM_ICFilter = 0xF;
//极性是否反转，并不是高低电平是否有效
TIM_ICInitstructure.TIM_ICPolarity =TIM_ICPolarity_Rising;
TIM_ICInit(TIM2,&TIM_ICInitstructure);
```

*/配置编码器模式,两个都采用上升沿模式/*
//配置IC极性，和IC结构体重复配置，配置极型是否反转
TIM_EncoderInterfaceConfig(TIM2,TIM_EncoderMode_TI12,ENCODER_A,ENCODER_B);

```
TIM_Cmd(TIM2,ENABLE);
```

}

# 电流环

需要改占空比，首先要进行ADC电流采样，采样完成后反帕克变换，变换结束后用PID进行调节输出给PWM一个占空比

在电流环实现前，需要对电角度d轴一个校准，把这个偏差值记录到编码器中，具体解释可以看我写的其他文档

FOC_ParkTransform(&parkTransform ,ADCSampling.Iu , ADCSampling.Iv,motorParameter.ElectricalDegree);
//PID调d轴校准值
DRV_MotorAlign(&Motor);
FOC_Pid(&pidId,parkTransform.Ds,Motor.alignId);
FOC_Pid(&pidIq,parkTransform.Qs,Motor.alignIq);
FOC_iParkTransform(&iparkTransform ,pidId.Out ,pidIq.Out ,motorParameter.ElectricalDegree);
FOC_SetSPWM(&Duty,iparkTransform.Alpha,iparkTransform.Beta);
Motor.stateNow = Motor.stateNext;

```
            //帕克变换
            FOC_ParkTransform(&parkTransform ,ADCSampling.Iu ,
ADCSampling.Iv,motorParameter.ElectricalDegree);
            //低通滤波 根据电机特性计算
            FOC_LowPassFilter(&lowPassFilterIq ,parkTransform.Qs);
            FOC_LowPassFilter(&lowPassFilterId,parkTransform.Ds);
            //PID调参 根据电机特性调参
            FOC_Pid(&pidId,lowPassFilterId.Out,Id);
            FOC_Pid(&pidIq,lowPassFilterIq.Out,Iq);
            FOC_iParkTransform(&iparkTransform ,pidId.Out
,pidIq.Out,motorParameter.ElectricalDegree);
            FOC_SetSPWM(&Duty,iparkTransform.Alpha,iparkTransform.Beta);
```

当然一些简单的滤波也是必要的

# 速度环

关键就是计算转速，用当前转速与需求转速进行PID调节，输出的值给电流环参考Iq就可以，其他基本不变

//帕克变换
FOC_ParkTransform(&parkTransform ,ADCSampling.Iu ,
ADCSampling.Iv,motorParameter.ElectricalDegree);
//低通滤波，采样Iq Id值
FOC_LowPassFilter(&lowPassFilterIq ,parkTransform.Qs);
FOC_LowPassFilter(&lowPassFilterId,parkTransform.Ds);
//转速PID输出到电流环参考值
FOC_Pid(&pidSpeed,MotorSpeed.Temp[0],Speed);
//转速输出PID值加入低通滤波，防止高频噪音
FOC_LowPassFilter(&lowPassFilterSpeedIq,pidSpeed.Out);
//PID调参　根据电机特性调参
FOC_Pid(&pidId,lowPassFilterId.Out,0);
FOC_Pid(&pidIq,lowPassFilterIq.Out,lowPassFilterSpeedIq.Out);

```
            FOC_iParkTransform(&iparkTransform ,pidId.Out
,pidIq.Out,motorParameter.ElectricalDegree);
            FOC_SetSPWM(&Duty,iparkTransform.Alpha,iparkTransform.Beta);
```

# 位置环

位置环最关键就是记录一个预计参考的脉冲值，与当前脉冲值做差值，形成PID再传参给速度环

//帕克变换
FOC_ParkTransform(&parkTransform ,ADCSampling.Iu ,
ADCSampling.Iv,motorParameter.ElectricalDegree);
//低通滤波，采样Iq Id值
FOC_LowPassFilter(&lowPassFilterIq ,parkTransform.Qs);
FOC_LowPassFilter(&lowPassFilterId,parkTransform.Ds);
//转速PID输出到电流环参考值
FOC_Pid(&pidSpeed,MotorSpeed.Temp[0],MotorPositon.Out);
//转速输出PID值加入低通滤波，防止高频噪音

FOC_LowPassFilter(&lowPassFilterSpeedIq,pidSpeed.Out);
//PID调参　根据电机特性调参
FOC_Pid(&pidId,lowPassFilterId.Out,0);
FOC_Pid(&pidIq,lowPassFilterIq.Out,lowPassFilterSpeedIq.Out);

```
            FOC_iParkTransform(&iparkTransform ,pidId.Out
,pidIq.Out,motorParameter.ElectricalDegree);
            FOC_SetSPWM(&Duty,iparkTransform.Alpha,iparkTransform.Beta);
```

# 电流采样，速度计算，位置计算

这几个才是最关键所在，

在电流采样要计算当前运算放大器的零点漂移现象，要确保α轴与β轴的电流值形成正弦波，这样才能配合电角度计算

**

- @brief 上电计算零点漂移值
- @param
- @example
- *
- 注意：
  */

static void DRV_AdcOffsetCalibration(void)
{
ADC_ExternalTrigInjectedConvEdgeConfig(ADC1, ADC_ExternalTrigInjecConvEdge_None);
//等待电流平缓
for(uint16_t i = 0 ; i<32 ; i++)
{

```
    ADC_SoftwareStartInjectedConv(ADC1);
    delay_1ms(1);
}
//求每个i的32次累加
for(uint16_t i = 0 ; i<3 ; i++)
{
    for(uint16_t j = 0 ; j<32 ;j++)
    {
        ADC_SoftwareStartInjectedConv(ADC1);
        ADCSampling.offsetIu[i] += ADC1->JDR1;
        ADCSampling.offsetIv[i] += ADC1->JDR2;
        delay_1ms(1);

    }
    //右移2的5次方=32 缩小32倍求平均
    ADCSampling.offsetIu[i]= ADCSampling.offsetIu[i]>>5;
    ADCSampling.offsetIv[i]= ADCSampling.offsetIv[i]>>5;
}
```

```
    ADCSampling.offsetMidIu = MID(ADCSampling.offsetIu[0],
ADCSampling.offsetIu[1], ADCSampling.offsetIu[2]);
    ADCSampling.offsetMidIv = MID(ADCSampling.offsetIv[0],
ADCSampling.offsetIv[1], ADCSampling.offsetIv[2]);
```

```
ADC_ExternalTrigInjectedConvEdgeConfig(ADC1,
ADC_ExternalTrigInjecConvEdge_Rising);
```

}

## 电角度计算

本项目所用电角度计算与其他不同，本文电角度计算是一共4000个AB信号，那么有50个极对数的步进电机，就是50个电角度周期，也就是80个AB信号就可以组成一个电角度周期，首先取到的电角度值取余80，对剩下的数值进行归一化变成0-360 转换 0-1，带入STM32自带DSP库中，sin 与 cos函数，因为这个三角函数是弧度值，所以需要稍微修改这个sin 与 cos函数，用归一化的值，再利用查表法查出sin 与 cos值，更加精确。

/**

- @brief 获取编码器机械角度转换成为电角度，再通过电角度归一化变成0-1区间浮点数方便计算
- @param
- @return 0-1归一化后电角度
- @example
- *
  *注意：
  *电机旋转一圈，编码器脉冲是PPR分辨率4000，对应360.0°/4000
  *电机转一圈，会有50个周期电角度，那么对应 PPR /POLE_PAIRS 4000/50 即80个计数值为一圈360.0°电角度
  *用当前计数值取余360°电角度的计数值即80，剩下的就是当前周期对应电角度
  *(count%( PPR / POLE_PAIRS ) )* (360.00f / (PPR / POLE_PAIRS));
  */

void app_ReadElectricalDegree(MOTOR_ParameterTypeDef*v)
{
  //获取当前计数值
  v->CurrentEncoder = TIM2->CNT;
  //计算差值
  v->DeltaCount = (int32_t)(v->CurrentEncoder - v->LastEncoder);
  //记录当前值给到下一时刻计算
  v->LastEncoder = v->CurrentEncoder;
  //处理溢出问题，假如反向溢出当前为3998 上一时刻为2，那么DeltaCount为3996，
  //那么需要加上减去周期4000，为正确增量
  if ((v->DeltaCount) > (int32_t)v->LineEncoder * 2)
  {
    v->DeltaCount =(v->DeltaCount) -(int32_t)(v->LinesPerTurn);
  }

//处理溢出问题，假如正向溢出当前为2 上一时刻为3998，那么DeltaCount为-3996，
//那么需要加上一个周期4000，为正确增量
if((v->DeltaCount) < -(int32_t)v->LineEncoder * 2)
{
  v->DeltaCount =(v->DeltaCount)+(int32_t)(v->LinesPerTurn);
}

```
v->RawCount += v->DeltaCount;
v->RawCount = v->RawCount % v->LinesPerTurn ;
if(v->RawCount < 0)
{
    v->RawCount=(v->RawCount)+(v->LinesPerTurn);
}
//待处理
v->MechanicalDegree = v->RawCount;
//计算电角度
//电角度计数值减去d轴偏差值
v->RawTheta = (v->CurrentEncoder) - (v->CalibratedDegree);
//d轴偏差为正值,当前值转换电角度记录不到比0-d的位置，需要补偿
if(v->RawTheta<0)
{
    v->RawTheta = v->RawTheta + v->LinesPerTurn;
}
//电角度偏差为负值,当前值转换电角度记录不到-d到0的区间，需要补偿
else if (v->RawTheta>v->LinesPerTurn)
{
    v->RawTheta=v->RawTheta - v->LinesPerTurn;
}
//计算电角度
v->ElectricalDegree = ((float)(v->RawTheta % v->LinesPerPoles)) * v-
>ElectricalScaler;
//用于
v->ElectricalSpeedDegree = v->ElectricalDegree;

//  //转换成机械角度4000个脉冲一圈 分出360度
//  v->MechanicalDegree = count * (360.0f /PPR) ;
//  //转换成电角度
//  v->ElectricalDegree = (count%( PPR / POLE_PAIRS ) )* (360.00f / (PPR /
POLE_PAIRS));
//



}
```

通过这些就可以实现电流环。

# 速度环

本文速度环，的计算是基于电角度增量，对间隔时间求导而完成的，也就是说本文有一个基准频率 这个频率是以1000rpm而转换出来的电角度频率，也就是以这个频率为基准0-1归一化统一单位。 也就是说本文的速度环是用电角度速度而完成，

/**

- @brief 通过电角度计算转速
- @param *v: DRV_MotorSpeedTypeDef结构体
- @example
- *
- 注意:
  */

void DRV_MotorSpeed(DRV_MotorSpeedTypeDef*v, float ElectricalSpeedDegree)
{
  v->SpeedCalCnt ++;
  if (v->SpeedCalCnt >=2)
  {
    v->SpeedCalCnt=0;

```
    v->ElecTheta = ElectricalSpeedDegree;
    //计算电角度差值
    v->DeltaTheta = (v->ElecTheta) -(v->OldElecTheta) ;
    //用于下一时刻计算差值
    v->OldElecTheta = v->ElecTheta;
    //判断差值是否超过一圈,于电角度计算同理
    if (v->DeltaTheta > (0.5f))
    {
        v->DeltaTheta = v->DeltaTheta - (1.0f);
    }
    else if (v->DeltaTheta < (-0.5f))
    {
        v->DeltaTheta = v->DeltaTheta + (1.0f);
    }
    //滤波
    v->DeltaTheta = (v->DeltaTheta)*(0.8f) + (v->OldDeltaTheta)*(0.2f);
    v->OldDeltaTheta = v->DeltaTheta;
    //计算转速M法 间隔时间内电机转过的角度
    //当前电角度旋转倒数差值 * 基波时间(1000rpm为基础) / T 可以获得转速
    v->Speed = (v->K1 * v->DeltaTheta);
    //低通滤波截止频率为20
    v->Speed = (v->OldSpeed * v->K2)+(v->Speed * v->K3);
    v->OldSpeed = v->Speed;

    //加权滤波
    v->Temp[0] = (v->Speed + v->Temp[1]*3.0f)*0.25f;
    v->Temp[1] = v->Temp[0];
    //实际转速*1000 是以1000为基波频率
    v->Temp[2] = v->Temp[0] * 1000.0f;

}
```

//转速参数计算
  MotorSpeed.K1=(1.0f / (BaseFreq * T * SpeedLoopPrescaler ));
  MotorSpeed.K2 =(1.0f/(1.0f + T *SpeedLoopPrescaler * 2 * PAI*20));
  MotorSpeed.K3 = (1.0f) - MotorSpeed.K2;

K2 K3是滤波参数,K1是基础频率，要是归一化，时间间隔是40us 求出这个速度，但是要进行归一化（具体解释看代码）

# 位置环

位置环也是要基于脉冲，并且进行归一化，否则三环单位不统一，无法精确调PID参数

/**

- @brief 通过脉冲计算当前位置与目标位置的偏差
- @param *v: DRV_MotorSpeedTypeDef结构体
- @example
- *
- 注意：自己理解简单运用 实际并不是这种方式
  */

void DRV_MotorPositon(DRV_MotorPositionTypeDef*v, int32_t ErrMechanicalDegree,int32_t TargetMechanicalDegree)
{
    v->PositonCalCnt ++;
    if (v->PositonCalCnt >=5)
    {
        //回零模式
        if(v->PositonMode == 0)
        {
            v->PositonCalCnt=0;
            //找Z点信号
            if(v->ZPositonFlag == 0)
            {
                v->PositonEncoderErr=10;
                v->Out = v->PositonEncoderErr v->Kp;
                v->Out = (float)(v->Out* v->PosScale);
                if (v->Out>= 0.005f )
                {
                    v->Out= 0.005f;
                }
                if (v->Out<= -0.005f )
                {
                    v->Out= -0.005f;
                }

```
            }
            //在Z信号处锁住
            if(v->ZPositonFlag == 1)
            {
                v->PositonEncoderNow = TIM2->CNT;
                v->PositonEncoderErr=v->PositonEncoderLast-v->PositonEncoderNow;
                v->PositonEncoderLast = v->PositonEncoderNow;
                v->Out = v->PositonEncoderErr* v->Kp;
                v->Out = (float)(v->Out* v->PosScale);
```

```
            if (v->Out>= 0.5f )
            {
                v->Out= 0.5f;
            }
            if (v->Out<= -0.5f )
            {
                v->Out= -0.5f;
            }
        }
```

```
    }
    //脉冲模式
    else if (v->PositonMode == 1)
    {
        v->ZPositonFlag =0;
        v->PositonCalCnt=0;
        //编码器增量,按下给脉冲标志位，给当前最新需求脉冲值
        if (v->PositonMoveFlag ==1  )
        {
            v->PositonEncoderTarget=0;
            v->PositonEncoderCNT=0;
            v->PositonEncoderTarget = TargetMechanicalDegree;
            v->PositonMoveFlag=0;
        }
        //累计当前脉冲，与需求脉冲做比较，*5是因为粗略计算5次电角度计算1次算脉冲
        v->PositonEncoderCNT += ErrMechanicalDegree*5;
        v->PositonEncoderCNT =  (v->PositonEncoderCNT)%  (v->EncoderCnt);
        v->Out = (v->PositonEncoderTarget- v->PositonEncoderCNT)* v->Kp;
        v->Out = (float)(v->Out* v->PosScale);
        if (v->Out>= 0.005f )
        {
            v->Out= 0.005f;
        }

        if (v->Out<= -0.005f )
        {
            v->Out= -0.005f;
        }

        //当累计脉冲和目标脉冲一致后，锁住电机
        if (v->PositonEncoderCNT == v->PositonEncoderTarget)
        {

            v->PositonEncoderNow = TIM2->CNT;
            v->PositonEncoderErr=v->PositonEncoderLast-v->PositonEncoderNow;
            v->PositonEncoderLast = v->PositonEncoderNow;
            v->Out = v->PositonEncoderErr* v->Kp;
            v->Out = (float)(v->Out* v->PosScale);
            if (v->Out>= 0.5f )
            {
                v->Out= 0.05f;
            }
```

```
            if (v->Out<= -0.05f )
            {
                v->Out= -0.05f;
            }

        }
```

```
    }
```

```
}
```

}

//增量的形式 增量/4000 取出0-1的范围，再除以时间系数1000转为基准频率的电角度增量频率
MotorPositon.PosScale =((STEP_POS_SCALE)/(4000*BaseFreq));
MotorPositon.Kp = 150;